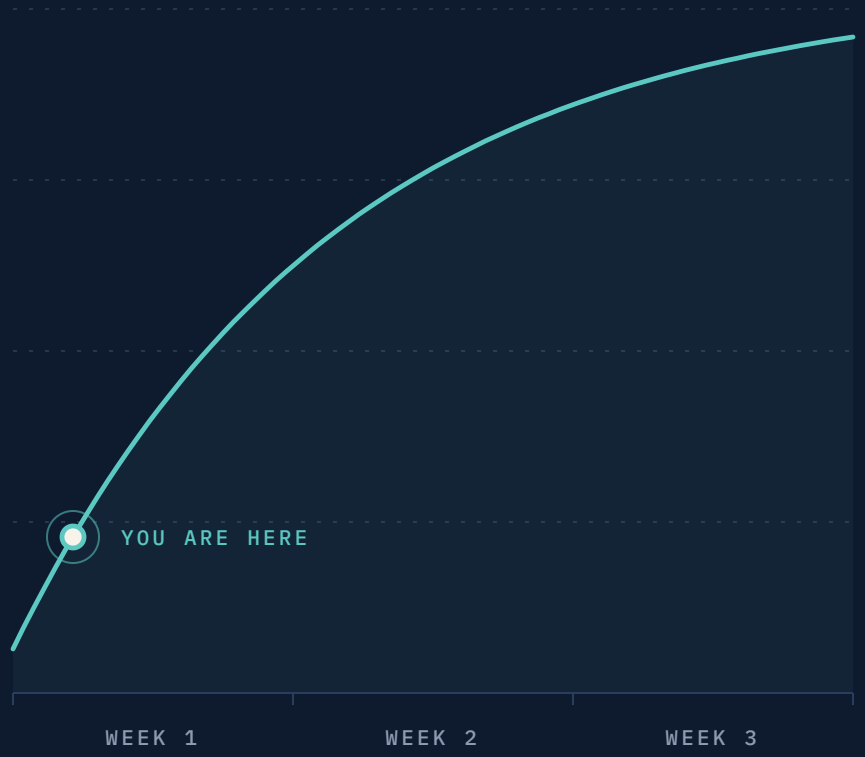
WEEK 1 · DAY 2 · OVERVIEW

Multiple Regression and Dummy Variables



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Day 1's rating line missed a typical unseen bond by 57.7 bp.

It also gave a AAA bond a fitted spread of -140 bp, because it takes the same 39 bp step at every notch.

The credit team now asks two questions.

Does a second column help the rating, and does the rating work better read as a label than as a number?

The bonds and issuers are invented for this course.

OBJECTIVES

Fit multiple linear regressions of OAS: on rating score and duration, on one column per rating and duration, then with sector added.

Read every coefficient as a difference from a level you name, score each model in bp on the 206 test bonds, and choose one to carry forward.

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

Your Route Through Day 2

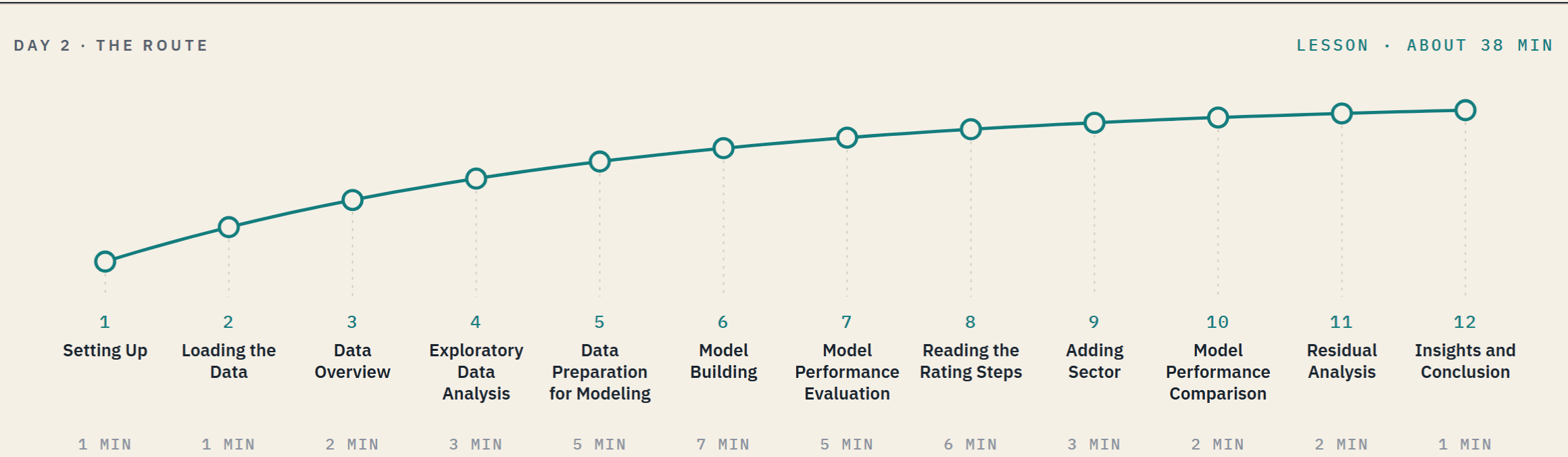


FIGURE 1 Twelve sections, in order, with about how long each takes.

Time. About 38 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SEED AND THE THREAD SETTINGS

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
```

THE LIBRARIES

```
# to load and work with the data
import numpy as np
import pandas as pd
# to draw charts
import matplotlib.pyplot as plt
import seaborn as sns
# to split the bonds, fit linear regressions, and score them
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
# one look for every chart, and plain numbers with three decimals in every table
sns.set_theme(style="whitegrid")
sns.set_palette("colorblind")
plt.rcParams["figure.figsize"] = (10, 5)
pd.set_option("display.float_format", "{:,.3f}".format)
```

Read the 821 bonds and the rating scale.

Loading the Data

READ AND JOIN THE FILES

```
bonds = pd.read_csv(DATA + "benchmark.csv")
ratings = pd.read_csv(DATA + "ratings.csv")
data = bonds.merge(ratings[["rating", "score", "grade"]], on="rating")
```

THE SECOND FILE

ratings.csv adds **score** (1 for AAA up to 18 for CCC) and **grade** (Investment Grade or High Yield).

WHAT EACH COLUMN HOLDS

- **cusip**: the bond's identifier
- **issuer**: the issuer's name
- **sector**: one of 10 industry sectors, used for the first time today
- **rating**: credit rating, AAA to CCC
- **oas**: option-adjusted spread, bp, **the target**
- **duration**: modified duration, years
- **amount_outstanding**: face value outstanding, dollars (Exercise 1)

Data Overview

THE FIRST ROWS

	CUSIP	ISSUER	SECTOR	RATING	OAS
0	99001BAA4	Quorvane Energy	Energy	BBB	83
1	99001BAB2	Quorvane Energy	Energy	BBB	122
2	99001BAC0	Quorvane Energy	Energy	BBB	123
3	99001BAD8	Quorvane Energy	Energy	BBB	88
4	99001BAE6	Quorvane Energy	Energy	BBB	127

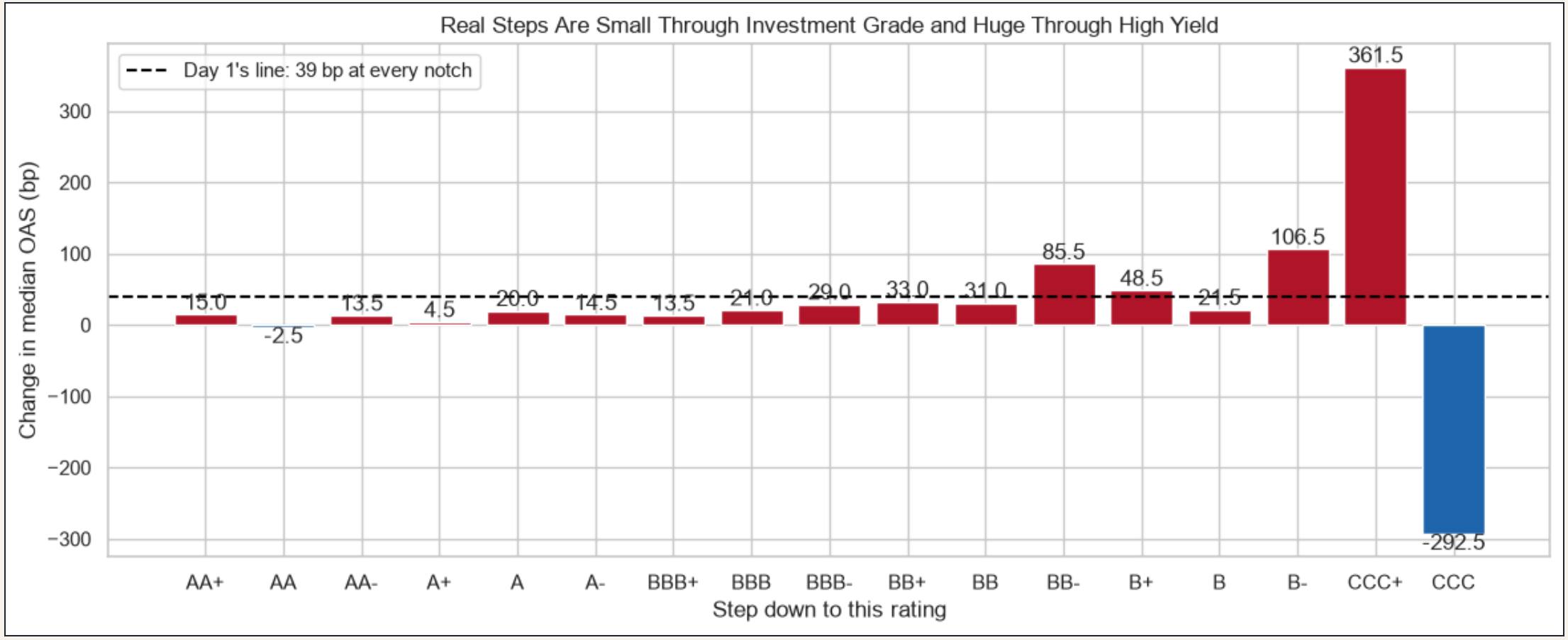
THE TEXT COLUMNS

```
print(data.shape)
data[["rating", "sector"]].nunique()
```

```
(821, 21)
rating    18
sector    10
dtype: int64
```

Both text columns go into a model today, so first we look at how spread moves across their levels.

Exploratory Data Analysis



Then the steps grow: 85.5 bp into BB-, 106.5 into B-, 361.5 into CCC+.

Data Preparation for Modeling

A text column becomes one indicator column per rating, minus the reference level

rating
AAA
A
BBB
A
BB

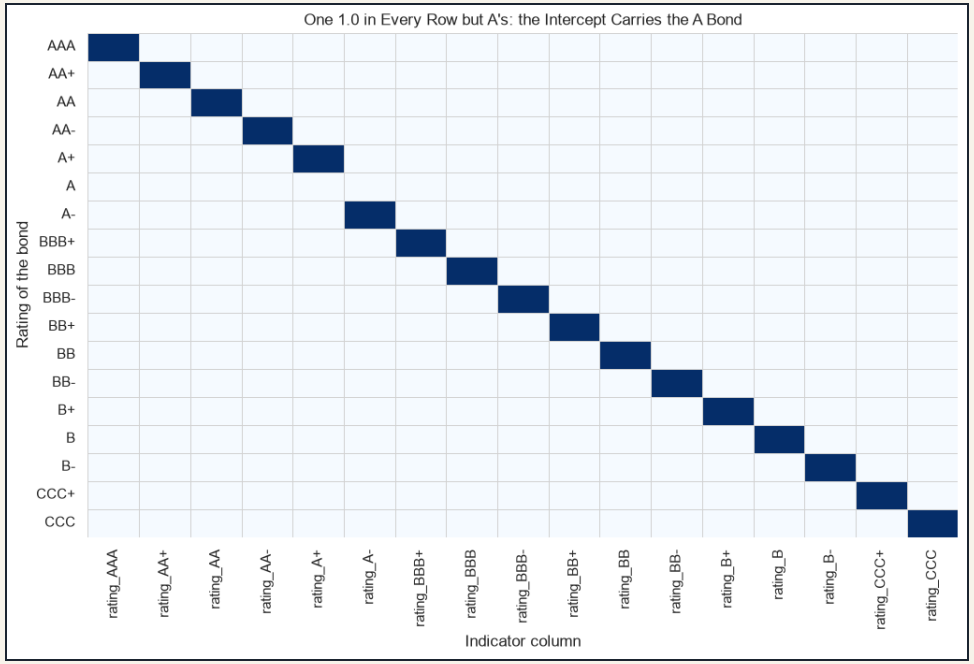
get_dummies →

rating_AAA	rating_A	rating_BBB	rating_BB
1.0	0.0	0.0	0.0
0.0	1.0	0.0	0.0
0.0	0.0	**1.0**	0.0
0.0	1.0	0.0	0.0
0.0	0.0	0.0	**1.0**

left out: A is the reference level, and the intercept stands for it

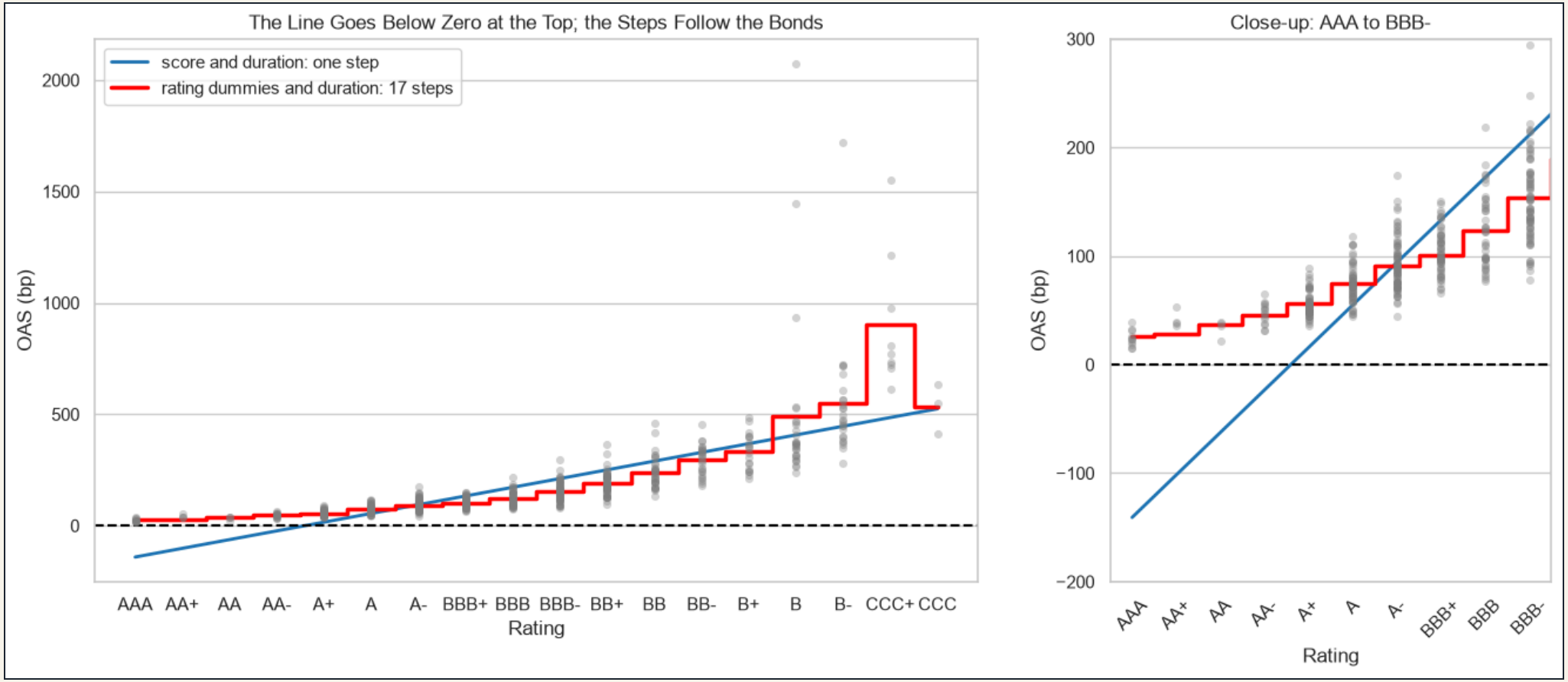
every other column's coefficient is a step measured from A

FIGURE 2 A rating column of text becomes one indicator column per rating, with A left out as the reference level.



Every column is now a number, and the test bonds are put away exactly as on Day 1.

Model Building



The steps follow the bonds. Now both models face the test bonds.

Score both models on bonds they never saw.

Model Performance Evaluation

TWO OF DAY 1'S FIVE MEASURES, AS CODE

```
# MAPE: each miss as a percent of the actual spread, averaged
def mape_score(actual, fitted):
    return np.mean(np.abs(actual - fitted) / actual) * 100
# adjusted R-squared: R-squared with a charge for each feature
def adj_r2_score(predictors, actual, fitted):
    r2 = r2_score(actual, fitted)
    n = predictors.shape[0]  # number of bonds
    k = predictors.shape[1]  # number of features
    return 1 - (1 - r2) * (n - 1) / (n - k - 1)
```

FITTED ON THE TRAINING BONDS: SCORE AND DURATION

	RMSE	MAE	MAPE	R-SQUARED
0	133.017	65.646	56.483	0.523

FITTED ON THE TRAINING BONDS: RATING DUMMIES AND DURATION

	RMSE	MAE	MAPE	R-SQUARED
0	110.177	42.710	21.219	0.673

Reading the Rating Steps

A STEP IS A DIFFERENCE OF AVERAGES

```
train_averages = y_train.groupby(train_rating).mean()
print("BBB coefficient:              ", round(bbb_step, 2))
print("BBB average minus A average: ", round(train_averages["BBB"] - train_averages["A"], 2))
print("Intercept:", round(model_ratings_only.intercept_, 2), " A average:", round(train_averages["A"], 2))
```

```
BBB coefficient:              48.99
BBB average minus A average:  48.99
Intercept: 74.6  A average: 74.6
```

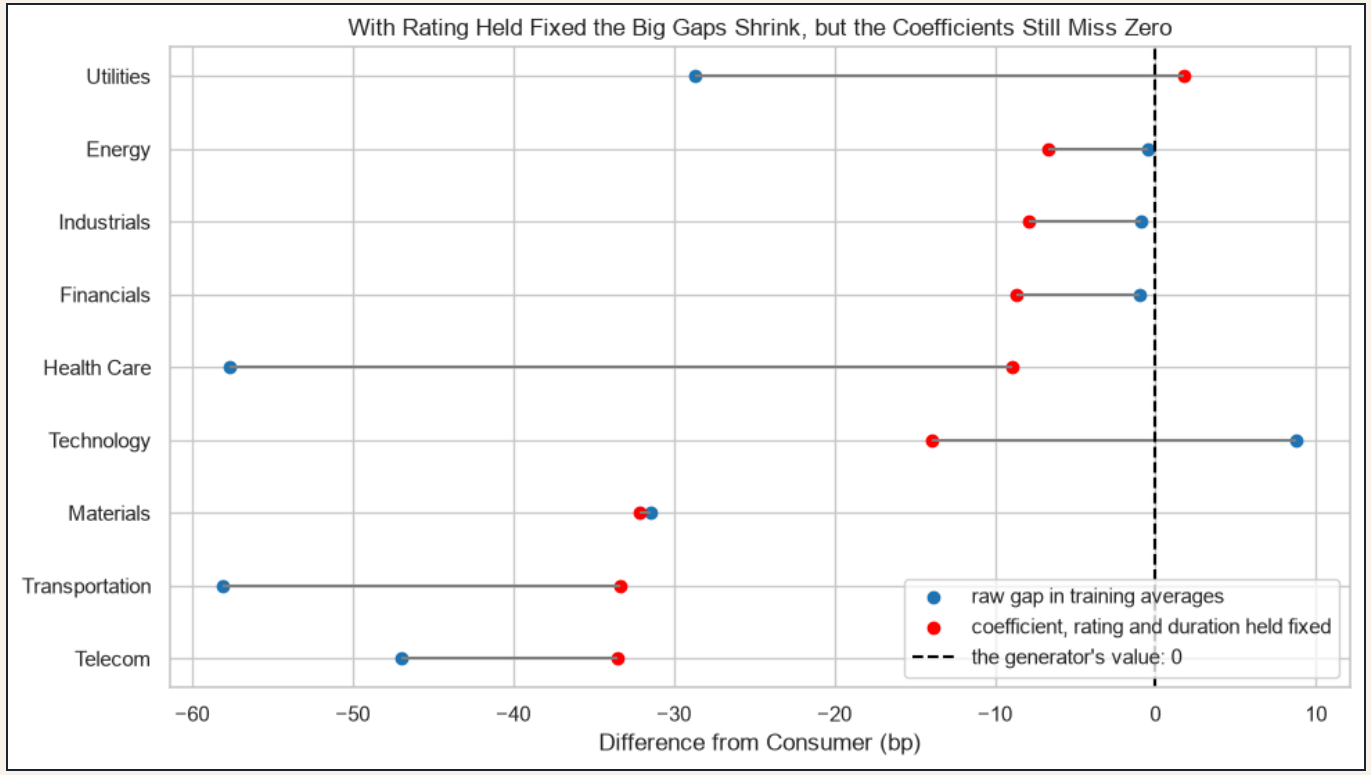
THE SHORTCUT LEAVES A LEVEL OUT FOR YOU

```
shortcut = pd.get_dummies(data["rating"], prefix="rating", drop_first=True)
print(list(shortcut.columns[:4]), len(shortcut.columns), "columns of", shortcut.dtypes.iloc[0])
```

```
['rating_A+', 'rating_A-', 'rating_AA', 'rating_AA+'] 17 columns of bool
```

Name the reference in your own code, with `.drop(columns=...)`, and say it whenever you quote a step.

Adding Sector



EACH SECTOR AGAINST CONSUMER

SECTOR	RAW GAP	HELD FIXED
Telecom	-46.926	-33.541
Transportation	-58.062	-33.329
Materials	-31.419	-32.146
Technology	8.796	-13.903
Health Care	-57.676	-8.888
Financials	-0.988	-8.696
Industrials	-0.910	-7.869
Energy	-0.488	-6.641
Utilities	-28.690	1.760

Model Performance Comparison

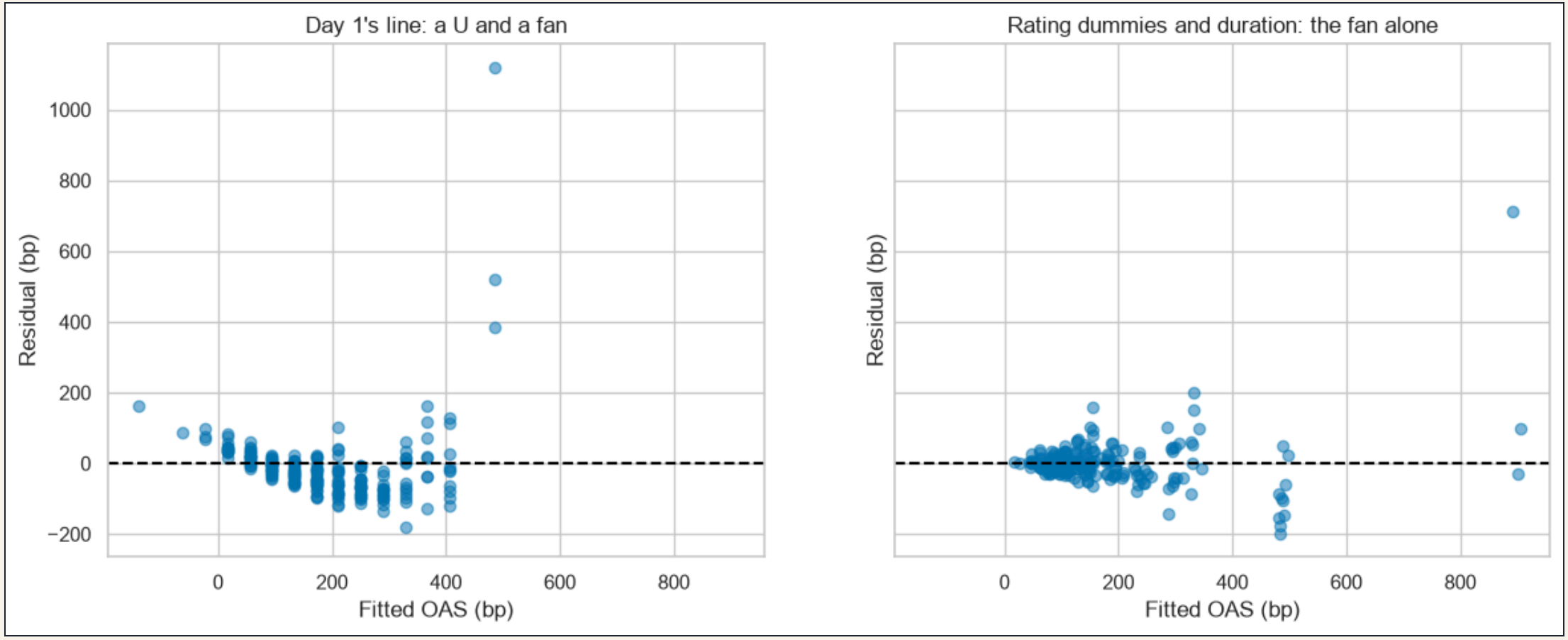
TRAINING PERFORMANCE, FOUR MODELS SIDE BY SIDE

	DAY 1'S LINE	SCORE AND DURATION	RATING DUMMIES AND DURATION	WITH SECTOR
RMSE	133.046	133.017	110.177	109.492
MAE	65.624	65.646	42.710	44.156
MAPE	56.365	56.483	21.219	23.734
R-squared	0.523	0.523	0.673	0.677
Adj. R-squared	0.522	0.521	0.663	0.662

ONE COLUMN PER MODEL

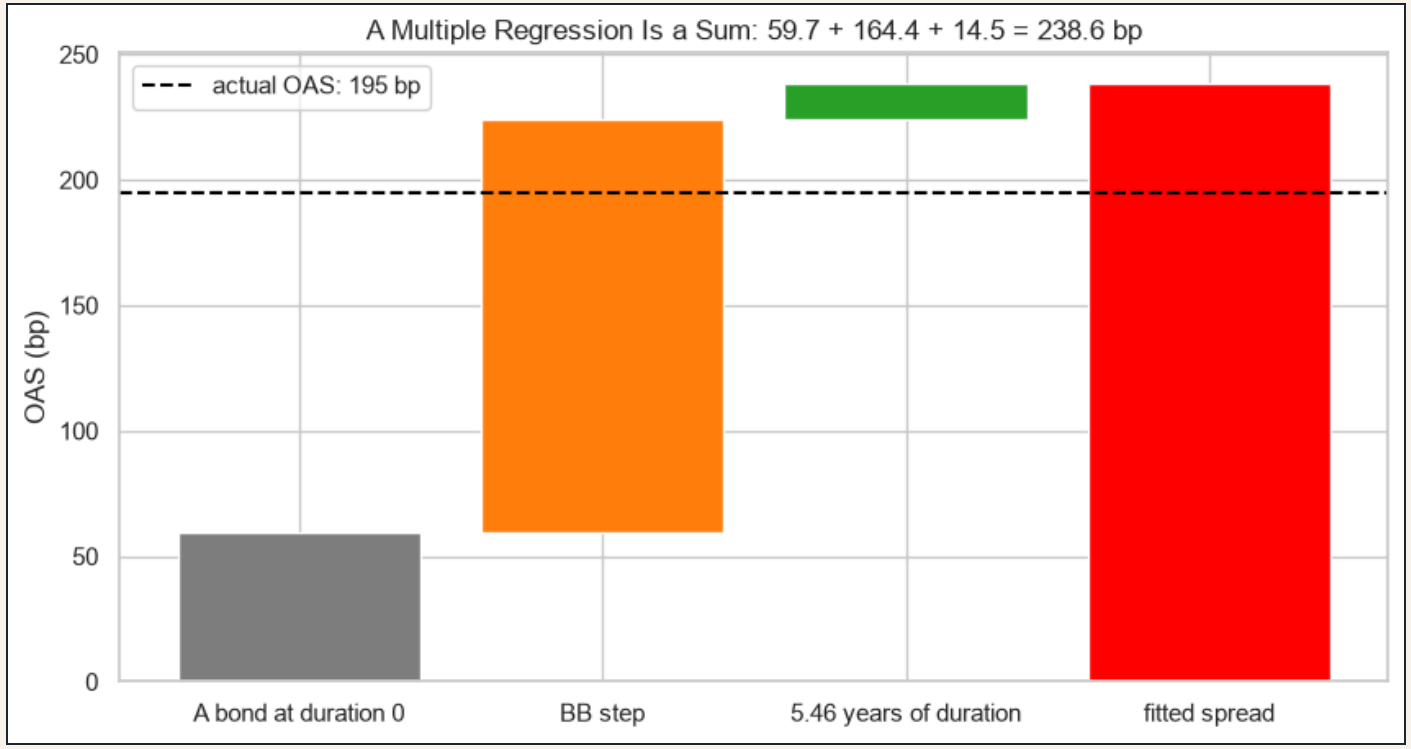
```
names = ["Day 1's line", "Score and duration", "Rating dummies and duration", "With sector"]
models_train_comp = pd.concat([line_train_perf.T, score_train_perf.T, steps_train_perf.T, sector_train_perf.T], axis=1)
models_train_comp.columns = names
models_train_comp
```

Residual Analysis



The shape is fixed; the fan is not, because the generator scales a bond's noise with its spread. Day 4 takes logs.

Insights and Conclusion



ONE BOND'S FITTED SPREAD, IN THREE PARTS

```
A bond at duration 0      59.653
BB step                  164.409
5.46 years of duration    14.535
dtype: float64
```

Every coefficient today was read by eye; Day 3 asks which of them can be told apart from zero.

You Can Now

- Fit a multiple linear regression and read each coefficient with the other features held fixed.
- Turn a text column into dummy variables with **`pd.get_dummies`** and name the reference level yourself.
- Read a rating step as a difference from the reference, and change the reference without changing a fitted spread.
- Spot the dummy trap and the level **`drop_first=True`** leaves out.
- Compare models on test bonds with MAE, an ECDF, and adjusted R-squared.

NEXT
WEEK 1 · DAY 3

Coefficients, P-Values, and Multicollinearity

Day 3 asks which coefficients can be told apart from zero.